

MD5

Материал из Википедии — свободной энциклопедии

MD5 (англ. *Message Digest 5*) — 128-битный алгоритм хеширования, разработанный профессором Рональдом Л. Ривестом из Массачусетского технологического института (Massachusetts Institute of Technology, MIT) в 1991 году. Предназначен для создания «отпечатков» или дайджестов сообщения произвольной длины и последующей проверки их подлинности. Широко применялся для проверки целостности информации и хранения хешей паролей.

Содержание

История

Алгоритм MD5

- Шаг 1. Выравнивание потока
- Шаг 2. Добавление длины сообщения
- Шаг 3. Инициализация буфера
- Шаг 4. Вычисление в цикле
- Шаг 5. Результат вычислений
- Псевдокод
- Сравнение MD5 и MD4

Примеры MD5-хешей

Криптоанализ

- Атаки переборного типа
- Коллизии MD5
 - Метод Ван Сяюня и Юй Хунбо

Примеры использования

Примечания

Литература

- Ссылки

MD5

Создан	<u>1991 г.</u>
Опубликован	апрель <u>1992 г.</u>
Предшественник	<u>MD4</u>
Преемник	<u>SHA-2</u>
Стандарты	<u>RFC 1321</u>
Размер хеша	128 бит
Число раундов	4
Тип	<u>хеш-функция</u>

История

MD5 — один из серии алгоритмов по построению дайджеста сообщения, разработанный профессором Рональдом Л. Ривестом из Массачусетского технологического института. Был разработан в 1991 году как более надёжный вариант предыдущего алгоритма MD4^[1].

Описан в [RFC 1321](#)^[2]. Позже [Гансом Доббертином](#) были найдены недостатки алгоритма MD4.

В 1993 году Берт ден Бур (Bert den Boer) и Антон Босселарс (Antoon Bosselaers) показали, что в алгоритме возможны псевдоколлизии, когда разным инициализирующим векторам соответствуют одинаковые дайджесты для входного сообщения^[3].

В 1996 году Ганс Доббертин (Hans Dobbertin) объявил о коллизии в алгоритме^[4], и уже в то время было предложено использовать другие алгоритмы хеширования, такие как [Whirlpool](#), [SHA-1](#) или [RIPEMD-160](#).

Из-за небольшого размера хеша в 128 бит можно рассматривать [birthday-атаки](#). В марте 2004 года был запущен проект MD5CRK с целью обнаружения уязвимостей алгоритма, при помощи [birthday-атаки](#). Проект MD5CRK закончился 17 августа 2004 года, когда Ван Сяюнь (Wang Xiaoyun), Фэн Дэньго (Feng Dengguo), Лай Сюэцзя (Lai Xuejia) и Юй Хунбо (Yu Hongbo) обнаружили уязвимости в алгоритме^[5].

1 марта 2005 года [Арьен Ленстра](#), Ван Сяюнь и Бенне де Вегер продемонстрировали построение двух документов [X.509](#) с различными открытыми ключами и одинаковым хешем MD5^[6].

18 марта 2006 года исследователь [Властимил Клима](#) (Vlastimil Klima) опубликовал алгоритм, который может найти коллизии за одну минуту на обычном компьютере, метод получил название «[туннелирование](#)»^[7].

В конце 2008 года [US-CERT](#) призвал разработчиков программного обеспечения, владельцев веб-сайтов и пользователей прекратить использовать MD5 в любых целях, так как исследования продемонстрировали ненадёжность этого алгоритма^[8].

24 декабря 2010 года Тао Се (Tao Xie) и Фэн Дэньго (Feng Dengguo) впервые представили коллизию сообщений длиной в один блок (512 бит)^[9]. Ранее коллизии были найдены для сообщений длиной в два блока и более. Позднее Марк Стивенс (Marc Stevens) повторил успех, опубликовав блоки с одинаковым хешем MD5, а также алгоритм для получения таких коллизий^[10].

В 2011 году был опубликован информационный документ [RFC 6151](#). Он признаёт алгоритм хеширования MD5^[2] небезопасным для некоторых целей и рекомендует отказаться от его использования в пользу SHA-2.

Алгоритм MD5

На вход алгоритма поступает входной поток данных, хеш которого необходимо найти. Длина сообщения измеряется в битах и может быть любой (в том числе нулевой). Запишем длину сообщения в *L*. Это число целое и неотрицательное. Кратность каким-либо числам необязательна. После поступления данных идёт процесс подготовки потока к вычислениям.

Ниже приведены 5 шагов алгоритма^[2]:

Шаг 1. Выравнивание потока

Сначала к концу потока дописывают единичный бит.

Затем добавляют некоторое число нулевых бит такое, чтобы новая длина потока L' стала сравнима с 448 по модулю 512, ($L' = 512 \times N + 448$). Выравнивание происходит в любом случае, даже если длина исходного потока уже сравнима с 448.

Шаг 2. Добавление длины сообщения

В конец сообщения дописывают 64-битное представление длины данных (количество бит в сообщении) до выравнивания. **Сначала записывают младшие 4 байта, затем старшие.** Если длина превосходит $2^{64} - 1$, то дописывают только младшие биты (эквивалентно взятию по модулю 2^{64}). После этого длина потока станет кратной 512. Вычисления будут основываться на представлении этого потока данных в виде массива слов по 512 бит.

Шаг 3. Инициализация буфера

Для вычислений инициализируются четыре переменные размером по 32 бита, начальные значения которых задаются шестнадцатеричными числами (порядок байтов little-endian):

```
A = 01 23 45 67; // 67452301h
B = 89 AB CD EF; // EFCDAB89h
C = FE DC BA 98; // 98BADCFEh
D = 76 54 32 10. // 10325476h
```

В этих переменных будут храниться результаты промежуточных вычислений. Начальное состояние ABCD называется инициализирующим вектором.

Шаг 4. Вычисление в цикле

Определим функции и константы, которые понадобятся нам для вычислений.

- Для каждого раунда потребуется своя функция. Введём функции от трёх параметров — слов, результатом также будет слово:

1-й этап: $\text{FunF}(X, Y, Z) = (X \wedge Y) \vee (\neg X \wedge Z)$,

2-й этап: $\text{FunG}(X, Y, Z) = (X \wedge Z) \vee (\neg Z \wedge Y)$,

3-й этап: $\text{FunH}(X, Y, Z) = X \oplus Y \oplus Z$,

4-й этап: $\text{FunI}(X, Y, Z) = Y \oplus (\neg Z \vee X)$,

где $\oplus, \wedge, \vee, \neg$ побитовые логические операции XOR, AND, OR и NOT соответственно.
- Определим таблицу констант $T[1 \dots 64]$ — 64-элементная таблица данных, построенная следующим образом: $T[n] = \text{int}(2^{32} \cdot |\sin n|)$.^[11]

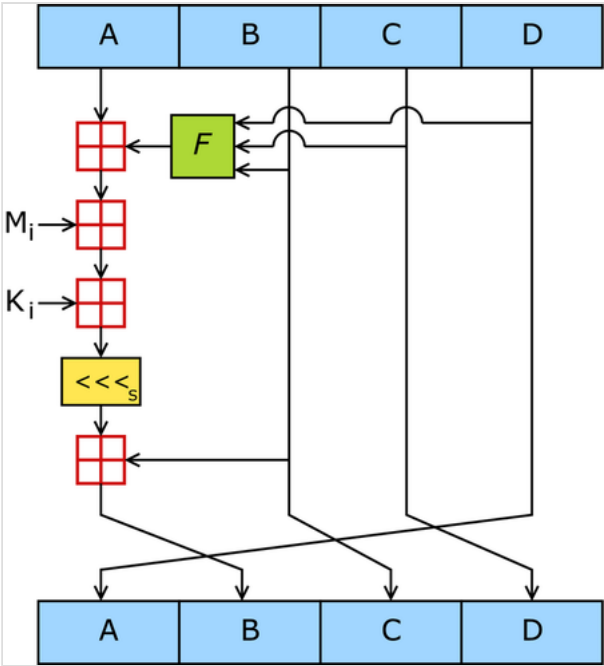


Схема работы алгоритма MD5. F — нелинейная функция. M_i обозначает 32-битный блок входного сообщения, а K_i — 32-битную константу. $\ll_s$ обозначает циклический сдвиг влево на s бит. $\boxplus$ обозначает сложение по модулю 2^{32} . F зависит от раунда, K_i и s меняются каждую операцию.

- Каждый 512-битный блок проходит 4 этапа вычислений по 16 раундов. Для этого блок представляется в виде массива X из 16 слов по 32 бита. Все раунды однотипны и имеют вид: $[abcd\ k\ s\ i]$, определяемый как $a = b + ((a + \text{Fun}(b, c, d) + X[k] + T[i]) \lll s)$, где k — номер 32-битного слова из текущего 512-битного блока сообщения, и $\dots \lll s$ — циклический сдвиг влево на s бит полученного 32-битного аргумента. Число s задается отдельно для каждого раунда.

Заносим в блок данных элемент n из массива 512-битных блоков. Сохраняются значения A , B , C и D , оставшиеся после операций над предыдущими блоками (или их начальные значения, если блок первый).

AA = A

BB = B

CC = C

DD = D

Этап 1

```
/* [abcd k s i] a = b + ((a + F(b,c,d) + X[k] + T[i]) <<< s). */
[ABCD 0 7 1][DABC 1 12 2][CDAB 2 17 3][BCDA 3 22 4]
[ABCD 4 7 5][DABC 5 12 6][CDAB 6 17 7][BCDA 7 22 8]
[ABCD 8 7 9][DABC 9 12 10][CDAB 10 17 11][BCDA 11 22 12]
[ABCD 12 7 13][DABC 13 12 14][CDAB 14 17 15][BCDA 15 22 16]
```

Этап 2

```
/* [abcd k s i] a = b + ((a + G(b,c,d) + X[k] + T[i]) <<< s). */
[ABCD 1 5 17][DABC 6 9 18][CDAB 11 14 19][BCDA 0 20 20]
[ABCD 5 5 21][DABC 10 9 22][CDAB 15 14 23][BCDA 4 20 24]
[ABCD 9 5 25][DABC 14 9 26][CDAB 3 14 27][BCDA 8 20 28]
[ABCD 13 5 29][DABC 2 9 30][CDAB 7 14 31][BCDA 12 20 32]
```

Этап 3

```
/* [abcd k s i] a = b + ((a + H(b,c,d) + X[k] + T[i]) <<< s). */
[ABCD 5 4 33][DABC 8 11 34][CDAB 11 16 35][BCDA 14 23 36]
[ABCD 1 4 37][DABC 4 11 38][CDAB 7 16 39][BCDA 10 23 40]
[ABCD 13 4 41][DABC 0 11 42][CDAB 3 16 43][BCDA 6 23 44]
[ABCD 9 4 45][DABC 12 11 46][CDAB 15 16 47][BCDA 2 23 48]
```

Этап 4

```
/* [abcd k s i] a = b + ((a + I(b,c,d) + X[k] + T[i]) <<< s). */
[ABCD 0 6 49][DABC 7 10 50][CDAB 14 15 51][BCDA 5 21 52]
[ABCD 12 6 53][DABC 3 10 54][CDAB 10 15 55][BCDA 1 21 56]
[ABCD 8 6 57][DABC 15 10 58][CDAB 6 15 59][BCDA 13 21 60]
[ABCD 4 6 61][DABC 11 10 62][CDAB 2 15 63][BCDA 9 21 64]
```

Суммируем с результатом предыдущего цикла:

```
A = AA + A
B = BB + B
C = CC + C
D = DD + D
```

После окончания цикла необходимо проверить, есть ли ещё блоки для вычислений. Если да, то переходим к следующему элементу массива ($n + 1$) и повторяем цикл.

Шаг 5. Результат вычислений

Результат вычислений находится в буфере ABCD, это и есть хеш. Если выводить побайтово, начиная с младшего байта A и заканчивая старшим байтом D, то мы получим MD5-хеш. 1, 0, 15, 34, 17, 18...

Псевдокод

```
// Все переменные – 32-битные беззнаковые целые. Все сложения выполняются по модулю  $2^{32}$ .

var int s[64], K[64]
var int i

// s обозначает величины сдвигов для каждой операции:
s[0..15] := { 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22 }
s[16..31] := { 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20 }
s[32..47] := { 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23 }
s[48..63] := { 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21 }

// Определяем таблицу констант следующим образом
for i from 0 to 63 do
    K[i] := floor( $2^{32} \times \text{abs}(\sin(i + 1))$ )
end for
// (Или просто используем заранее подсчитанные значения):
K[0..3] := { 0xd76aa478, 0xe8c7b756, 0x242070db, 0xc1bdceee }
K[4..7] := { 0xf57c0faf, 0x4787c62a, 0xa8304613, 0xfd469501 }
K[8..11] := { 0x698098d8, 0xb444f7af, 0xfffff5bb1, 0x895cd7be }
K[12..15] := { 0x6b901122, 0xfd987193, 0xa679438e, 0x49b40821 }
K[16..19] := { 0xf61e2562, 0xc040b340, 0x265e5a51, 0xe9b6c7aa }
K[20..23] := { 0xd62f105d, 0x02441453, 0xd8a1e681, 0xe7d3fbc8 }
K[24..27] := { 0x21e1cde6, 0xc33707d6, 0xf4d50d87, 0x455a14ed }
K[28..31] := { 0xa9e3e905, 0xfcefa3f8, 0x676f02d9, 0x8d2a4c8a }
K[32..35] := { 0xffffa3942, 0x8771f681, 0x6d9d6122, 0xfde5380c }
K[36..39] := { 0xa4beea44, 0x4bdecfa9, 0xf6bb4b60, 0xbebfbcb70 }
K[40..43] := { 0x289b7ec6, 0xeaa127fa, 0xd4ef3085, 0x04881d05 }
K[44..47] := { 0xd9d4d039, 0xe6db99e5, 0x1fa27cf8, 0xc4ac5665 }
K[48..51] := { 0xf4292244, 0x432aff97, 0xab9423a7, 0xfc93a039 }
K[52..55] := { 0x655b59c3, 0x8f0ccc92, 0xffefff47d, 0x85845dd1 }
K[56..59] := { 0x6fa87e4f, 0xfe2ce6e0, 0xa3014314, 0x4e0811a1 }
K[60..63] := { 0xf7537e82, 0xbd3af235, 0x2ad7d2bb, 0xeb86d391 }

// Инициализация переменных:
var int a0 := 0x67452301 // A
var int b0 := 0xefcdab89 // B
var int c0 := 0x98badcfe // C
var int d0 := 0x10325476 // D

// Подготовка: добавляем бит "1" в конец сообщения.
append "1" bit to message
// Заметка: входные байты представлены строкой из бит,
// причем первый бит – старший (big-endian).

// Подготовка: дописываем нулевые биты, пока длина сообщения не станет сравнима с 448 по модулю 512
append "0" bit until message length in bits ≡ 448 (mod 512)
// Допишем остаток от деления изначальной длины сообщения на  $2^{64}$ 
append original length in bits mod  $2^{64}$  to message

// Разбиваем подготовленное сообщение на 512-битные "кусочки":
for each 512-bit chunk of padded message do
    // и работаем с каждым по отдельности
    break chunk into sixteen 32-bit words M[j],  $0 \leq j \leq 15$  // разбиваем "кусочек" на 16 блоков по 32 бита
    // Инициализируем переменные для текущего куска:
    var int A := a0
    var int B := b0
    var int C := c0
    var int D := d0
    // Основные операции:
    for i from 0 to 63 do
```

```

var int F, g
if 0 ≤ i ≤ 15 then
    F := (B and C) or ((not B) and D)
    g := i
else if 16 ≤ i ≤ 31 then
    F := (D and B) or ((not D) and C)
    g := (5×i + 1) mod 16
else if 32 ≤ i ≤ 47 then
    F := B xor C xor D
    g := (3×i + 5) mod 16
else if 48 ≤ i ≤ 63 then
    F := C xor (B or (not D))
    g := (7×i) mod 16
F := F + A + K[i] + M[g] // M[g] — 32 битный блок
A := D
D := C
C := B
B := B + (F <<< s[i]) // Выполняем битовый сдвиг
end for
// Прибавляем результат текущего "куска" к общему результату
a0 := a0 + A
b0 := b0 + B
c0 := c0 + C
d0 := d0 + D
end for

var char digest[16] := a0 append b0 append c0 append d0 // (Результат в формате little-endian)

```

Результат вычислений на примере языка программирования Python

```

1  import hashlib
2  import math
3  k:int
4  r:int
5  def left_rotate(n, b):
6      return ((n << b) | (n >> (32 - b))) & 0xffffffff
7
8  def md5(message):
9      a0 = 0x67452301
10     b0 = 0xefcdab89
11     c0 = 0x98badcfe
12     d0 = 0x10325476
13
14     m1 = len(message) * 8
15     message = bytearray(message)
16     message.append(0x80)
17
18     while len(message) % 64 != 56:
19         message.append(0)
20
21     message += m1.to_bytes(8, byteorder='little')
22
23     for i in range(0, len(message), 64):
24         chunk = message[i:i+64]
25
26         a = a0
27         b = b0
28         c = c0
29         d = d0
30
31         # Main Loop
32         for j in range(64):
33             if 0 <= j <= 15:
34                 f = (b & c) | ((~b) & d)
35                 g = j
36             elif 16 <= j <= 31:
37                 f = (d & b) | ((~d) & c)
38                 g = (5*j + 1) % 16
39             elif 32 <= j <= 47:
40                 f = b ^ c ^ d
41                 g = (3*j + 5) % 16
42             elif 48 <= j <= 63:
43                 f = c ^ (b | (~d))
44                 g = (7*j) % 16
45
46         d_temp = d

```

```

47         d = c
48         c = b
49         b = (b + left_rotate((a + f + k[j] + int.from_bytes(chunk[4*g:4*(g+1)], byteorder='little')),
r[j])) & 0xffffffff
50         a = d_temp
51
52         a0 = (a0 + a) & 0xffffffff
53         b0 = (b0 + b) & 0xffffffff
54         c0 = (c0 + c) & 0xffffffff
55         d0 = (d0 + d) & 0xffffffff
56
57     return '{:08x}{:08x}{:08x}{:08x}'.format(a0, b0, c0, d0)
58
59 # Пример использования:
60 import hashlib
61 message = "Hello, world!"
62 result = hashlib.md5(message.encode()).hexdigest()
63 print("MD5 хэш (hashlib):", result)
64
65 def md5(message):
66     # Constants
67     T = [int(abs(math.sin(i+1)) * 2**32) & 0xFFFFFFFF for i in range(64)]
68     s = [[7, 12, 17, 22]] * 4 + [[5, 9, 14, 20]] * 4 + [[4, 11, 16, 23]] * 4 + [[6, 10, 15, 21]] * 4
69
70     # Initialize variables
71     A, B, C, D = 0x67452301, 0xEFCDAB89, 0x98BADCFE, 0x10325476
72     message = bytearray(message)
73     length = (8 * len(message)) & 0xFFFFFFFFFFFFFFFF
74     message.append(0x80)
75     while len(message) % 64 != 56:
76         message.append(0x00)
77     message.extend(length.to_bytes(8, byteorder='little'))
78
79     # Process message in 16-word blocks
80     for i in range(0, len(message), 64):
81         X = [int.from_bytes(message[i:i+4], byteorder='little') for i in range(i, i+64, 4)]
82         A_, B_, C_, D_ = A, B, C, D
83
84         # Main Loop
85         for j in range(64):
86             if j < 16:
87                 F = (B & C) | ((~B) & D)
88                 F_index = j
89             elif j < 32:
90                 F = (D & B) | ((~D) & C)
91                 F_index = (5 * j + 1) % 16
92             elif j < 48:
93                 F = B ^ C ^ D
94                 F_index = (3 * j + 5) % 16
95             else:
96                 F = C ^ (B | (~D))
97                 F_index = (7 * j) % 16
98
99             dTemp = D
100             D = C
101             C = B
102             B = B + left_rotate((A + F + T[j] + X[F_index]) & 0xFFFFFFFF, s[j % 4][j % 4])
103             A = dTemp
104
105         # Update state
106         A = (A + A_) & 0xFFFFFFFF
107         B = (B + B_) & 0xFFFFFFFF
108         C = (C + C_) & 0xFFFFFFFF
109         D = (D + D_) & 0xFFFFFFFF
110
111     # Output
112     result = bytearray(A.to_bytes(4, byteorder='little'))
113     result.extend(B.to_bytes(4, byteorder='little'))
114     result.extend(C.to_bytes(4, byteorder='little'))
115     result.extend(D.to_bytes(4, byteorder='little'))
116     return result.hex()
117
118 result = md5(message.encode())
119 print("MD5 хэш (самостоятельная реализация):", result)

```

Сравнение MD5 и MD4

Алгоритм MD5 происходит от MD4. В новый алгоритм добавили ещё один раунд, теперь их стало 4 вместо 3 в MD4. Добавили новую константу для того, чтобы свести к минимуму влияние входного сообщения, в каждом раунде на каждом шаге и каждый раз константа разная, она суммируется с результатом F и блоком данных. Изменилась функция $G = XZ \vee (Y \neg Z)$ вместо $XY \vee XZ \vee YZ$. Результат каждого шага складывается с результатом предыдущего шага, из-за этого происходит более быстрое изменение результата. Для этой же цели оптимизирована величина сдвига на каждом круге. Изменился порядок работы с входными словами в раундах 2 и 3^[2].

Примеры MD5-хешей

Хеш содержит 128 бит (16 байт) и обычно представляется как последовательность из 32 шестнадцатеричных цифр^[12].

Несколько примеров хеша:

`MD5("md5") = 1BC29B36F623BA82AAAF6724FD3B16718`

Даже небольшое изменение входного сообщения (в нашем случае на один бит: ASCII символ «5» с кодом 35₁₆ = 00011010**1**₂ заменяется на символ «4» с кодом 34₁₆ = 00011010**0**₂) приводит к полному изменению хеша. Такое свойство алгоритма называется лавинным эффектом.

`MD5("md4") = C93D3BF7A7C4AFE94B64E30C2CE39F4F`

Пример MD5-хеша для «нулевой» строки:

`MD5("") = D41D8CD98F00B204E9800998ECF8427E`

Криптоанализ

На данный момент существуют несколько видов «взлома» хешей MD5 — подбора сообщения с заданным хешем^{[13][14]}:

- Перебор по словарю
- Brute-force
- RainbowCrack
- Коллизия хеш-функции

При этом методы перебора по словарю и brute-force могут использоваться для взлома хеша других хеш-функций (с небольшими изменениями алгоритма). В отличие от них, RainbowCrack требует предварительной подготовки радужных таблиц, которые создаются для заранее определённой хеш-функции. Поиск коллизий специфичен для каждого алгоритма.

Атаки переборного типа

Для полного перебора или перебора по словарю можно использовать программы PasswordsPro^[15], MD5BFCPF^[16], John the Ripper, Hashcat. Для перебора по словарю существуют готовые словари^[17]. Основным недостатком такого типа атак является высокая вычислительная сложность.

RainbowCrack — ещё один метод нахождения прообраза хеша из заданного множества. Он основан на генерации цепочек хешей, чтобы по получившейся базе вести поиск заданного хеша. Хотя создание радужных таблиц занимает много времени и памяти, последующий взлом производится очень быстро. Основная идея данного метода — достижение компромисса между временем поиска по таблице и занимаемой памятью.

Коллизии MD5

Коллизия хеш-функции — это получение одинакового значения функции для разных сообщений и идентичного начального буфера. В отличие от коллизий, *псевдоколлизии* определяются как равные значения хеша для разных значений начального буфера, причём сами сообщения могут совпадать или различаться. В MD5 вопрос коллизий не решается^[14].

В 1996 году Ганс Доббертин нашёл псевдоколлизии в MD5, используя определённые *инициализирующие* векторы, отличные от стандартных. Оказалось, что можно для известного сообщения построить второе, такое, что оно будет иметь такой же хеш, как и исходное. С точки зрения математики это означает: MD5(IV,L1) = MD5(IV,L2), где IV — начальное значение буфера, а L1 и L2 — различные сообщения. Например, если взять начальное значение буфера^[4]:

A = 0x12AC2375

B = 0x3B341042

C = 0x5F62B97C

D = 0x4BA763E

и задать входное сообщение

AA1DDABE	D97ABFF5	BBF0E1C1	32774244
1006363E	7218209D	E01C136D	9DA64D0E
98A1FB19	1FAE44B0	236BB992	6B7A779B
1326ED65	D93E0972	D458C868	6B72746A

то, добавляя число **2⁹** к определённому 32-разрядному слову в блочном буфере, можно получить второе сообщение с таким же хешем. Ханс Доббертин представил такую формулу:

$$L2_i = \begin{cases} L1_i, & i \neq 14; \\ L1_i + 2^9, & i = 14. \end{cases}$$

Тогда MD5(IV, L1) = MD5(IV, L2) = BF90E670752AF92B9CE4E3E1B12CF8DE.

В 2004 году китайские исследователи Ван Сяюнь (Wang Xiaoyun), Фэн Дэнго (Feng Dengguo), Лай Сюэцзя (Lai Xuejia) и Юй Хунбо (Yu Hongbo) объявили об обнаруженной ими уязвимости в алгоритме, позволяющей за небольшое время (1 час на кластере IBM p690) находить коллизии^{[5][18]}.

В 2005 году Ван Сяюнь и Юй Хунбо из университета Шаньдуна в Китае опубликовали алгоритм, который может найти две различные последовательности в 128 байт, которые дают одинаковый MD5-хеш. Одна из таких пар (различающиеся разряды выделены):

d131dd02c5e6eec4693d9a0698aff95c	2fcab58712467eab4004583eb8fb7f89
55ad340609f4b30283e488832571415a	085125e8f7cdc99fd91dbdf280373c5b
d8823e3156348f5bae6dacd436c919c6	dd53e2b487da03fd02396306d248cda0
e99f33420f577ee8ce54b67080a80d1e	c69821bcb6a8839396f9652b6ff72a70

и

d131dd02c5e6eec4693d9a0698aff95c	2fcab50712467eab4004583eb8fb7f89
55ad340609f4b30283e4888325f1415a	085125e8f7cdc99fd91dbd7280373c5b
d8823e3156348f5bae6dacd436c919c6	dd53e23487da03fd02396306d248cda0
e99f33420f577ee8ce54b67080280d1e	c69821bcb6a8839396f965ab6ff72a70

Каждый из этих блоков даёт MD5-хеш, равный 79054025255fb1a26e4bc422aef54eb4^[19].

В 2006 году чешский исследователь Властимил Клима опубликовал алгоритм, позволяющий находить коллизии на обычном компьютере с любым начальным вектором (A,B,C,D) при помощи метода, названного им «туннелирование»^{[7][20]}.

Алгоритм MD5 использует итерационный метод Меркла — Дамгора, поэтому становится возможным построение коллизий с одинаковым, заранее выбранным префиксом. Аналогично, коллизии получаются при добавлении одинакового суффикса к двум различным префиксам, имеющим одинаковый хеш. В 2009 году было показано, что для любых двух заранее выбранных префиксов можно найти специальные суффиксы, с которыми сообщения будут иметь одинаковое значение хеша. Сложность такой атаки составляет всего 2³⁹ операций подсчёта хеша^[21].

Метод Ван Сяюня и Юй Хунбо

Метод Ван Сяюня и Юй Хунбо использует тот факт, что MD5 построен на итерационном методе Меркла — Дамгора. Поданный на вход файл сначала дополняется, так чтобы его длина была кратна 64 байтам, после этого он делится на блоки по 64 байта каждый $M_0, M_1, \dots, M_{n-1}$. Далее вычисляется последовательность 16-байтных состояний $s_0, \dots, s_n$ по правилу $s_{i+1} = f(s_i, M_i)$, где f — некоторая фиксированная функция. Начальное состояние s_0 называется инициализирующим вектором.

Метод позволяет для заданного инициализирующего вектора найти две пары M, M' и N, N' , такие что $f(f(s, M), M') = f(f(s, N), N')$. Этот метод работает для любого инициализирующего вектора, а не только для вектора используемого по стандарту.

Эта атака является разновидностью дифференциальной атаки, которая, в отличие от других атак этого типа, использует целочисленное вычитание, а не XOR в качестве меры разности. При поиске коллизий используется метод модификации сообщений: сначала выбирается произвольное сообщение M_0 , далее оно модифицируется по некоторым правилам, сформулированным в статье, после чего вычисляется дифференциал хеш-функции, причём $M'_0 = M_0 + dM_0$ с вероятностью 2^{-37} . К M_0 и M'_0 применяется функция сжатия для проверки условий коллизии; далее выбирается произвольное M_1 , модифицируется,

вычисляется новый дифференциал, равный нулю с вероятностью 2^{-30} , а равенство нулю дифференциала хеш-функции как раз означает наличие коллизии. Оказалось, что найдя одну пару M_0 и M'_0 , можно менять лишь два последних слова в M_0 , тогда для нахождения новой пары M_1 и M'_1 требуется всего около 2^{39} операций хеширования^[19].

Применение этой атаки к MD4 позволяет найти коллизию меньше чем за секунду. Она также применима к другим хеш-функциям, таким как RIPEMD и HAVAL^[5].

Примеры использования

Ранее считалось, что MD5 позволяет получать относительно надёжный идентификатор для блока данных. На данный момент данная хеш-функция не рекомендуется к использованию, так как существуют способы нахождения коллизий с приемлемой вычислительной сложностью^{[14][22]}.

Свойство уникальности хеша широко применяется в разных областях^[23]. Приведенные примеры относятся и к другим криптографическим хеш-функциям.

С помощью MD5 проверяли целостность и подлинность скачанных файлов — так, некоторые программы поставляются вместе со значением контрольной суммы. Например, пакеты для инсталляции свободного ПО^[24].

MD5 использовался для хеширования паролей. В системе UNIX каждый пользователь имеет свой пароль и его знает только пользователь. Для защиты паролей используется хеширование. Предполагалось, что получить настоящий пароль можно только полным перебором. При появлении UNIX единственным способом хеширования был DES (Data Encryption Standard), но им могли пользоваться только жители США, потому что исходные коды DES нельзя было вывозить из страны. Во FreeBSD решили эту проблему. Пользователи США могли использовать библиотеку DES, а остальные пользователи имеют метод, разрешённый для экспорта. Поэтому в FreeBSD стали использовать MD5 по умолчанию.^[25] Некоторые Linux-системы также используют MD5 для хранения паролей^[26].

Многие системы используют базы данных для аутентификации пользователей и существует несколько способов хранения паролей^[27]:

1. Пароли хранятся как есть. При взломе такой базы все пароли станут известны.
2. Хранятся только хеши паролей. Найти пароли можно, используя заранее подготовленные таблицы хешей. Такие таблицы составляются из хешей простых или популярных паролей.
3. К каждому паролю добавляется несколько случайных символов (их называют «соль») и результат хешируется. Полученный хеш вместе с «солью» сохраняются в открытом виде. Найти пароль с помощью таблиц таким методом не получится.

Существует несколько надстроек над MD5.

- MD5 (HMAC) — Keyed-Hashing for Message Authentication (хеширование с ключом для аутентификации сообщения) — алгоритм позволяет хешировать входное сообщение L с некоторым ключом K, такое хеширование позволяет аутентифицировать подпись^[28].
- MD5 (Base64) — здесь полученный MD5-хеш кодируется алгоритмом Base64.

- MD5 (Unix) — алгоритм вызывает тысячу раз стандартный MD5 для усложнения процесса. Также известен как MD5crypt^[29].

Примечания

1. What are MD2, MD4, and MD5? (<https://www.webcitation.org/61AADFziE?url=http://www.rsa.com/rsalabs/node.asp?id=2253>) (англ.). RSA Laboratories (2000). Дата обращения: 11 июля 2009. Архивировано из оригинала (<http://www.rsa.com/rsalabs/node.asp?id=2253>) 23 августа 2011 года.
2. Rivest, 1992.
3. Boer, Bosselaers, 1993.
4. *Hans Dobbertin*. The Status of MD5 After a Recent Attack (<ftp://ftp.arnes.si/packages/crypto-tools/rsa.com/cryptobytes/crypto2n2.pdf.gz>). Дата обращения: 22 октября 2015.
5. *Xiaoyun Wang, Dengguo Feng, Xuejia Lai, Hongbo Yu*. Collisions for Hash Functions MD4, MD5, HAVAL-128 and RIPEMD (<https://www.webcitation.org/61AAF4PT7?url=http://eprint.iacr.org/2004/199>) (англ.) (17 августа 2004). Дата обращения: 19 ноября 2008. Архивировано из оригинала (<http://eprint.iacr.org/2004/199>) 23 августа 2011 года.
6. *Arjen Lenstra, Xiaoyun Wang and Benne de Weger*. Colliding X.509 Certificates (<http://eprint.iacr.org/2005/067.pdf>) . *eprint.iacr.org* (1 марта 2005). Дата обращения: 4 декабря 2015. Архивировано (<https://web.archive.org/web/20160304205816/http://eprint.iacr.org/2005/067.pdf>) 4 марта 2016 года.
7. *Vlastimil Kli'ma*. Tunnels in Hash Functions: MD5 Collisions Within a Minute (<https://www.webcitation.org/61AAFv2q7?url=http://eprint.iacr.org/2006/105>) (англ.) (17 апреля 2006). Дата обращения: 19 ноября 2008. Архивировано из оригинала (<http://eprint.iacr.org/2006/105>) 23 августа 2011 года.
8. CERT Vulnerability Note VU#836068 (<https://www.kb.cert.org/vuls/id/836068>) (англ.). kb.cert.org (30 декабря 2008). Дата обращения: 10 октября 2015. Архивировано (<https://web.archive.org/web/20110726144513/http://www.kb.cert.org/vuls/id/836068>) 26 июля 2011 года.
9. *Tao Xie, Dengguo Feng*. Construct MD5 Collisions Using Just A Single Block Of Message (<http://eprint.iacr.org/2010/643>) (PDF) (16 декабря 2010). Дата обращения: 16 октября 2015. Архивировано (<https://web.archive.org/web/20170514185806/http://eprint.iacr.org/2010/643>) 14 мая 2017 года.
10. Marc Stevens – Research – Single-block collision attack on MD5 (<http://marc-stevens.nl/research/md5-1block-collision/>). Marc-stevens.nl (2012). Дата обращения: 16 октября 2015. Архивировано (<https://web.archive.org/web/20170515070731/http://marc-stevens.nl/research/md5-1block-collision/>) 15 мая 2017 года.
11. Иными словами, в таблице представлены по 32 бита после десятичной запятой от значений функции *sin*, где аргумент *n* в радианах.
12. Detection Of Phishing Websites And Secure Transactions (https://web.archive.org/web/20160304092343/http://interscience.in/IJCNS_Vol1Iss2/paper3.pdf) . Anna University (2012). Дата обращения: 20 октября 2015. Архивировано из оригинала (http://interscience.in/IJCNS_Vol1Iss2/paper3.pdf) 4 марта 2016 года.
13. Ah Kioon, Wang, Deb Das, 2013.
14. Updated Security Considerations for the MD5 Message-Digest and the HMAC-MD5 Algorithms (<https://tools.ietf.org/html/rfc6151>). Internet Engineering Task Force (март 2011). Дата обращения: 23 октября 2015. Архивировано (<https://web.archive.org/web/20170615213134/https://tools.ietf.org/html/rfc6151>) 15 июня 2017 года.
15. PasswordsPro (<https://web.archive.org/web/20110827033811/http://www.insidepro.com/rus/passwordspro.shtml>). InsidePro Software. — Программа для восстановления паролей к хешам различных типов. Дата обращения: 19 ноября 2008. Архивировано из оригинала (<http://www.insidepro.com/rus/passwordspro.shtml>) 27 августа 2011 года.
16. Проект MD5 (<http://md5bfcpf.sourceforge.net>) на сайте SourceForge.net

17. CERIAS — Security Archive (<ftp://ftp.cerias.purdue.edu/pub/dict/>). Center for Education and Research in Information Assurance and Security (июнь 2000). Дата обращения: 19 ноября 2008. Архивировано (<https://web.archive.org/web/20081207033141/http://ftp.cerias.purdue.edu/pub/dict/>) 7 декабря 2008 года.
18. *Philip Hawkes, Michael Paddon, Gregory G. Rose*. Musings on the Wang et al. MD5 Collision (<https://www.webcitation.org/61AAFVLMw?url=http://eprint.iacr.org/2004/264>) (англ.) (13 октября 2004). Дата обращения: 19 ноября 2008. Архивировано из оригинала (<http://eprint.iacr.org/2004/264>) 23 августа 2011 года.
19. Wang, Yu, 2005.
20. *Vlastimil Klima*. MD5 collisions (https://www.webcitation.org/61AAGKcF4?url=http://cryptograph.y.hyperlink.cz/MD5_collisions.html) (англ.). Дата обращения: 19 ноября 2008. Архивировано из оригинала (http://cryptography.hyperlink.cz/MD5_collisions.html) 23 августа 2011 года.
21. Stevens, Lenstra, Weger, 2012.
22. *Marc Stevens, Arjen Lenstra and Benne de Weger*. Vulnerability of software integrity and code signing applications to chosen-prefix collisions for MD5 (<http://www.win.tue.nl/hashclash/SoftIntCodeSign/>) (30 ноября 2007). Дата обращения: 25 октября 2015. Архивировано (<https://web.archive.org/web/20071213023720/http://www.win.tue.nl/hashclash/SoftIntCodeSign/>) 13 декабря 2007 года.
23. *Ilya Mironov*. Hash functions: Theory, attacks, and applications (http://research.microsoft.com/pubs/64588/hash_survey.pdf) . *Microsoft Research* (14 ноября 2005). Дата обращения: 13 ноября 2015. Архивировано (https://web.archive.org/web/20160304022937/http://research.microsoft.com/pubs/64588/hash_survey.pdf) 4 марта 2016 года.
24. *Turnbull J.* Hardening Linux (англ.) — 1 — Apress, 2005. — P. 57—58.
25. *Bill Swingle*. Руководство FreeBSD (DES, MD5 и шифрование) (<https://web.archive.org/web/20090917074507/http://www.nbuu.gov.ua/books/2004/freebsd/crypt.html>) (2006). Дата обращения: 20 ноября 2008. Архивировано из оригинала (<http://www.nbuu.gov.ua/books/2004/freebsd/crypt.html>) 17 сентября 2009 года.
26. *Vicki Stanfield, Roderick W. Smith*. Linux System Administration (Craig Hunt Linux Library). — 2. — Sybex, 2002. — С. 479—483. — 656 с. — ISBN 978-0782141382.
27. *Hossein Bidgoli*. The Internet Encyclopedia, Volume 3. — 1. — Wiley, 2003. — С. 3—4. — 908 с. — ISBN 978-0471222019.
28. *Krawczyk, Hugo, Canetti, Ran, Bellare, Mihir*. HMAC: Keyed-Hashing for Message Authentication (<https://tools.ietf.org/html/rfc2104>). tools.ietf.org. Дата обращения: 5 декабря 2015. Архивировано (<https://web.archive.org/web/20210415003434/https://tools.ietf.org/html/rfc2104>) 15 апреля 2021 года.
29. *Steven Alexander*. password protection for modern operating systems (<http://c59951.r51.cf2.rackcdn.com/4902-1103-alexander.pdf>) . *USENIX 2004* (июнь 2004). Дата обращения: 5 декабря 2015. Архивировано (<https://web.archive.org/web/20151208104416/http://c59951.r51.cf2.rackcdn.com/4902-1103-alexander.pdf>) 8 декабря 2015 года.

Литература

- *Rivest R.* The MD5 Message-Digest Algorithm (<https://tools.ietf.org/html/rfc1321>) (англ.) — IETF, 1992. — 21 p. — doi:10.17487/RFC1321 (<https://dx.doi.org/10.17487/RFC1321>)
- *Boer B. d., Bosselaers A.* Collisions for the compression function of MD5 (http://link.springer.com/content/pdf/10.1007%2F3-540-48285-7_26.pdf) (англ.) // *Advances in Cryptology — EUROCRYPT '93: Workshop on the Theory and Application of Cryptographic Techniques Lofthus, Norway, May 23–27, 1993 Proceedings* / T. Hellesest — 1 — Berlin: Springer Berlin Heidelberg, 1993. — P. 293—304. — 465 p. — ISBN 978-3-540-57600-6 — doi:10.1007/3-540-48285-7_26 (https://dx.doi.org/10.1007/3-540-48285-7_26)
- *Xiaoyun W., Yu H.* How to Break MD5 and Other Hash Functions (http://link.springer.com/content/pdf/10.1007%2F11426639_2.pdf) (англ.) // *Advances in Cryptology — EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005. Proceedings* / R. Cramer — Springer

- Science+Business Media, 2005. — P. 19—35. — 578 p. — ISBN 978-3-540-25910-7 — doi:10.1007/11426639_2 (https://dx.doi.org/10.1007/11426639_2)
- *Stevens M., Lenstra A. K., Weger B. d.* Chosen-prefix collisions for MD5 and applications (<http://documents.epfl.ch/users/l/le/lenstra/public/papers/lat.pdf>) (англ.) // *International Journal of Applied Cryptography* — Inderscience Publishers, 2012. — Vol. 2, Iss. 4. — P. 322—359. — ISSN 1753-0563 (<https://www.worldcat.org/issn/1753-0563>); 1753-0571 (<https://www.worldcat.org/issn/1753-0571>) — doi:10.1504/IJACT.2012.048084 (<https://dx.doi.org/10.1504/IJACT.2012.048084>)
 - *Ah Kioon, Mary Cindy, Wang Z., Deb Das S.* Security Analysis of MD5 Algorithm in Password Storage (англ.) // *Applied Mechanics and Materials* — 2013. — Vol. 2706-2711. — ISSN 1660-9336 (<https://www.worldcat.org/issn/1660-9336>); 1662-7482 (<https://www.worldcat.org/issn/1662-7482>); 2297-8941 (<https://www.worldcat.org/issn/2297-8941>) — doi:10.4028/WWW.SCIENTIFIC.NET/AMM.347-350.2706 (<https://dx.doi.org/10.4028/WWW.SCIENTIFIC.NET/AMM.347-350.2706>)

Ссылки

- <https://www.md5.cz/> - Генератор MD5 хеша из обычного текста
- <https://www.md5online.org/> - База данных простых MD5 хешей (Позволяет произвести обратный декодирование простых MD5 хешей, используя базу данных сгенерированных хешей)
- <https://md5.kz/> - Одноименный образовательный центр в городе Алматы. Наименование образовательного центра является отсылкой на материал, который используется в нескольких лекциях по направлению WEB разработки (<https://md5.kz/?do=kurs-razrabotki-web-saitov>).

Источник — <https://ru.wikipedia.org/w/index.php?title=MD5&oldid=135369922>

Эта страница в последний раз была отредактирована 5 января 2024 в 18:08.

Текст доступен по лицензии Creative Commons «С указанием авторства — С сохранением условий» (CC BY-SA); в отдельных случаях могут действовать дополнительные условия.

Wikipedia® — зарегистрированный товарный знак некоммерческой организации Фонд Викимедиа (Wikimedia Foundation, Inc.)